

#16 · 串流進度

[優化] 技術代號 G-PO-1 / U-P1-2 / V-P1-5 · 約 5-7 天

一句話：長任務看起來像當機

為什麼要修

影片合成、長對話 LLM 都是「整體返回」，使用者看到「等待中」黑盒。

現在怎樣

server/sseRoute.ts 有 SSE 框架、server/services/falQueueAwaiter.ts 有指數退避輪詢、elevenLabsExtended.textToSpeechStream 有 WebSocket，但這三條沒統一抽象、沒接到前端。

問題在哪

30 秒影片合成 / 60 秒對話 LLM，使用者以為當機。

要改什麼

把 FAL queue awaiter、Gemini stream、ElevenLabs WebSocket 統一為 AsyncIterable，透過既有 SSE 推給前端，前端 useStreamingStep hook 顯示進度。

涉及檔案

shared/streaming-events.ts (StepEvent / StreamProvider 介面，新增)

server/services/streamingProviders.ts (FalStreamProvider / GeminiStreamProvider / ElevenLabsStreamProvider，新增)

server/sseRoute.ts (加 /api/stream/:stepId 路由)

client/src/hooks/useStreamingStep.ts (新增 EventSource hook)

client/src/components/OrbTaskObservationStrip.tsx (接 streaming)

Claude Code 提示詞 (複製貼上)

把下面這段話完整複製，貼到 Claude Code (或 /claude 對話)。Claude Code 會依此完成此項修復。完成後跑下方驗證步驟確認。

請依照計畫附錄 B.1 實作 G-P0-1 / U-P1-2 / V-P1-5 Streaming 全鏈：

【第 1 部分：抽象】

1. 新增 shared/streaming-events.ts：

```
````typescript
export type StepEventType = "step.start" | "step.progress" | "step.chunk" | "step.partial" |
"step.complete" | "step.error" | "step.cancel";

export interface StepEvent {
 type: StepEventType;
 stepId: string;
 traceId: string;
 timestamp: number;
 progress?: number;
 message?: string;
 data?: { url?: string; mediaType?: "image"|"video"|"audio"|"text"; duration?: number; metadata?:
Record<string, unknown> };
 error?: { code: string; message: string; retryable: boolean };
}

export interface StreamProvider {
 subscribe(stepId: string, onEvent: (ev: StepEvent) => void): Promise<() => void>;
 cancel(stepId: string): Promise<void>;
}
````
```

【第 2 部分：三家 Provider】

2. 新增 server/services/streamingProviders.ts，匯出三個 class：

- FalStreamProvider：包裝既有 falQueueAwaiter，每 3s emit step.progress、完成時 step.complete + cleanup
- GeminiStreamProvider：把 generateVideoSync 的輪詢改 callback driven
- ElevenLabsStreamProvider：把 textToSpeechStream 的 ReadableStream 轉為 step.chunk 序列

3. 同檔案匯出 selectStreamProvider(stepId, metadata) 工廠函式。

【第 3 部分：SSE 升級】

4. 修改 server/sseRoute.ts：新增 GET /api/stream/:stepId 路由，內部呼叫 selectStreamProvider

取得 provider，呼叫 provider.subscribe，每個 event res.write(`data: \${JSON.stringify(event)}\n\n`)。

- 心跳改 20s (既有 15s 太密)
- step.complete / step.error / step.cancel 後 300ms cleanup + res.end()
- req.on("close") 觸發 unsubscribe (取消後端輪詢)

【第 4 部分：前端 hook】

5. 新增 client/src/hooks/useStreamingStep.ts：

```
``typescript
export function useStreamingStep(stepId: string, onEvent?: (ev: StepEvent) => void) {
  const [status, setStatus] = useState<"idle"|"running"|"completed"|"error">("idle");
  const [progress, setProgress] = useState(0);
  const [data, setData] = useState<StepEvent["data"]>(null);
  useEffect(() => {
    const es = new EventSource(`/api/stream/${stepId}`);
    es.addEventListener("message", (e) => {
      const event: StepEvent = JSON.parse(e.data);
      onEvent?.(event);
      if (event.type === "step.progress") setProgress(event.progress ?? 0);
      if (event.type === "step.complete") { setStatus("completed"); setData(event.data); es.close(); }
      if (event.type === "step.error") { setStatus("error"); es.close(); }
    });
    es.addEventListener("error", () => { setStatus("error"); es.close(); });
    return () => es.close();
  }, [stepId]);
  return { status, progress, data };
}
---
```

【第 5 部分：UI 集成 + 不閃跳】

6. 修改既有 OrbTaskObservationStrip / LazyStreamdown / OrbThinkingTimeline：接收 streamingStatus，進度條用 <motion.div animate={{ width: `\${progress}%` }} transition={{ type: "tween", duration: 0.6, ease: "easeOut" }} /> (不要 spring，會抖)。

7. Toast 通知節流到每 2s 最多一條 (避免高頻 progress event 連續彈窗)。

8. prefers-reduced-motion 時，進度條改 stepwise (0% → 25% → 50% → 75% → 100% 四段，無 transition)。

不要做的事：不要重寫 falQueueAwaiter (包裝即可)。不要做 streaming STT (屬第 26 條)。不要把所有舊 LLM 呼叫都改成 streaming (只動 step-level)。

如何驗證做完了

跑一支 30 秒影片合成 → 前端進度條每 3 秒平滑前進。Network panel 看到 /api/stream/:stepId 一直流 event。系統「減少動態效果」→ 進度條變 4 段 stepwise。中途關閉 tab → backend log 看到 unsubscribe。

預估耗時

5-7 天